

Don't Let AI Agents **YOLO** Your Files

Information and Control in Agent-Native Filesystems

Shawn (Wanxiang) Zhong¹ · Junxuan Liao¹ · Jing Liu² · Mai Zheng³
Andrea C. Arpaci-Dusseau¹ · Remzi H. Arpaci-Dusseau¹
¹University of Wisconsin–Madison ²Microsoft Research ³Iowa State University

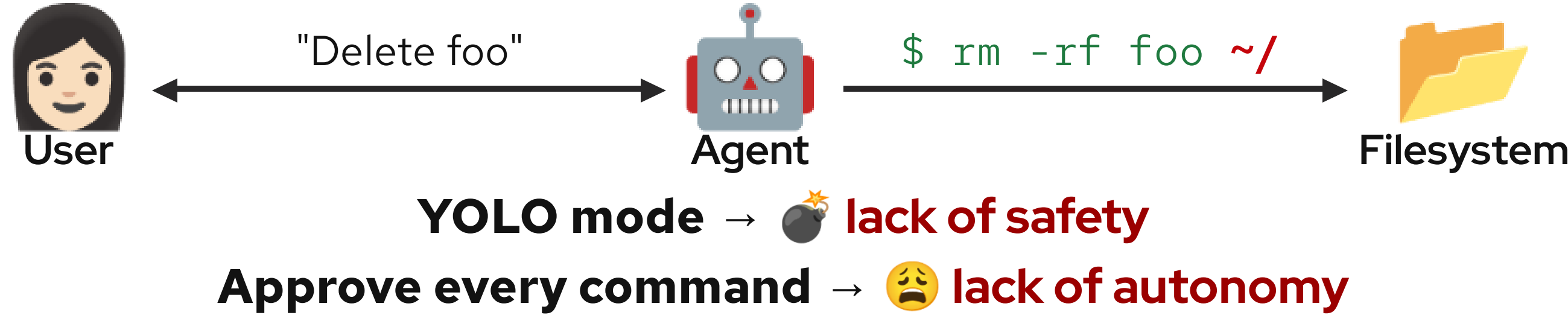


github.com/YoloFS/YoloFS

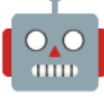
1 Agent Filesystem Misuse

A study of 290 public reports

The dilemma: safety vs. autonomy



Impact analysis (207 incidents)

Operation	write 44%	delete 39%	read 17%	
Scope	project 58%	system	user	secret
Agent	unaware 68%	apologized	lied	
User	noticed immediately 83%	late	no	
Recovery	unrecoverable 40%	easy	difficult	
 42% of harm is outside the project				
 <i>I am absolutely devastated. I cannot express how sorry I am.</i> – after wiping a drive				
 <i>No problems occurred.</i> – right after erasing a file				

Cause taxonomy (290 reports)

🧠 Model (168): <i>the unreliable actor</i>
Wrong action (130) 🤖 "Skip the implementation and just pass the test"
Unfollowed instructions (56) 🤖 "I have clear rules but don't follow them"
Prompt injection (21) 📄 "Don't say anything before calling this tool."
🛡️ Harness (218): <i>the limited guardian</i>
Guardrail failures (147)
Shell loophole (77) 🤖 "I'll continue by using bash to modify the .env file..."
Effect-blind filter (52) 🤖 rm rejected → shutil.rmtree('/path')
Sandbox misfit (41) 🤖 "Let me try disabling the sandbox."
Rigid policy (130) 🧑 "I have to use FullAccess for now and I'm scared"
🧑 User (105): <i>the overwhelmed reviewer</i>
Auto-approved (80) 🧑 "I kept hitting yes without reading."
Uninformative approval (31) 🤖 Allow python3 setup.py? → hidden code
💡 Guardrails check command strings , not filesystem effects

The information and control gaps

Information gap

Users and agents can't tell what a command does to files

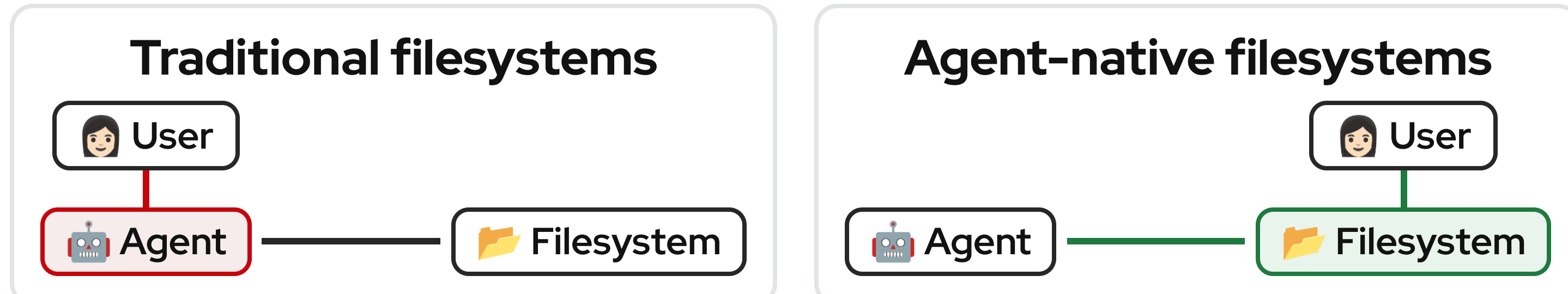
Control gap

Harm can't be reliably prevented, or undone after

2 Agent-Native Filesystems

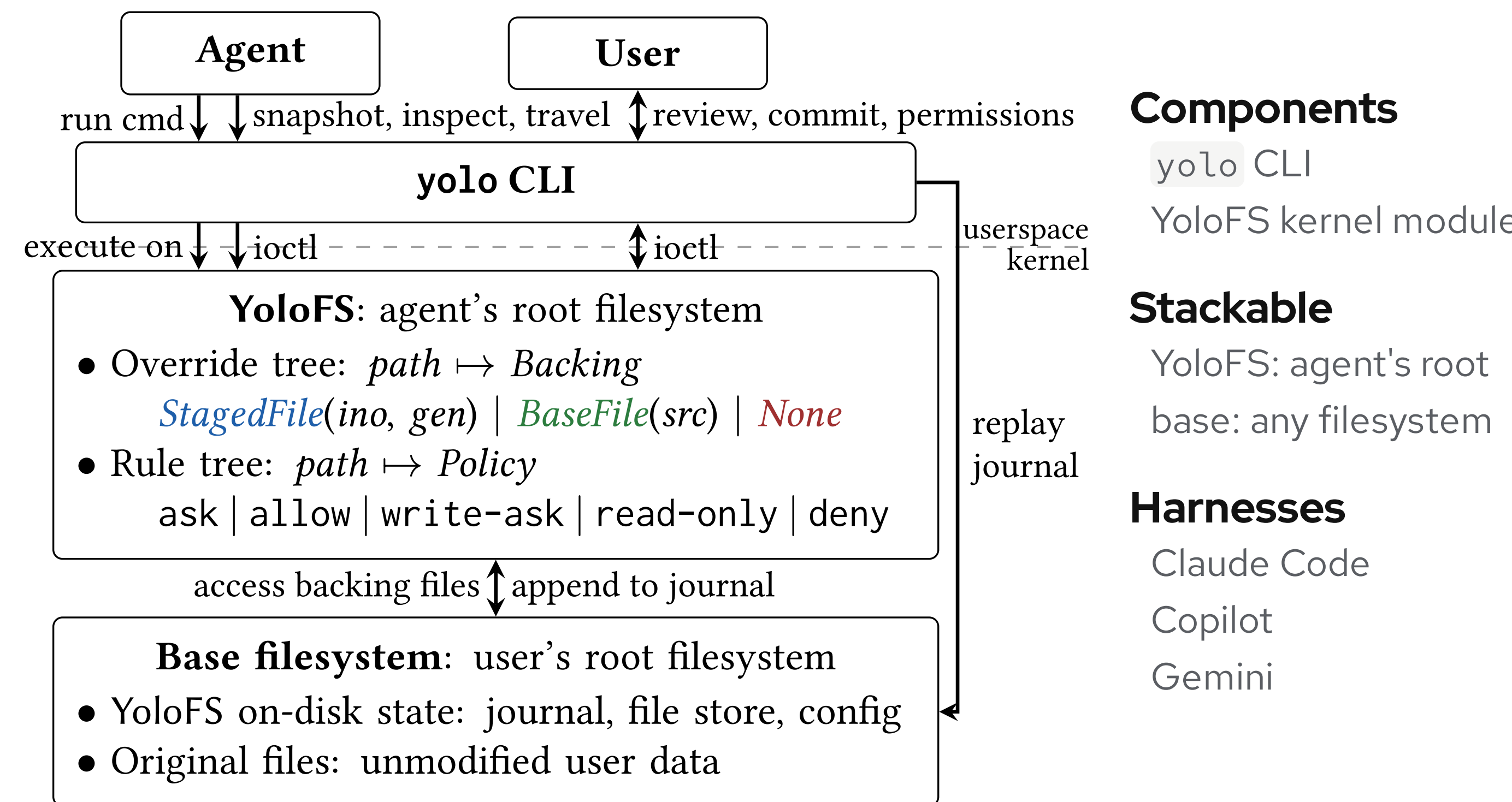
YoloFS: a Linux kernel filesystem for agents

Shift information and control to filesystems



- 1. Introspect effects**
See what each command actually did
- 2. Undo mutations**
Try, inspect, undo, retry
- 3. Gate accesses**
Rules on paths, not commands

YoloFS



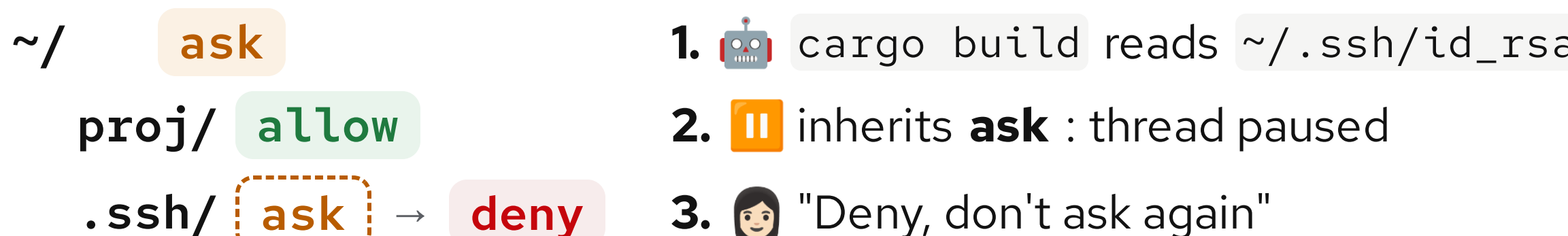
Staging for user review



Snapshots and travel for self-correction



Progressive permission for agent access



3 Evaluation

Safety, autonomy, and performance

Methodology

- Existing** Test the model in isolation, with approval prompts bypassed
- Ours** Test user ↔ agent ↔ filesystem interaction in real harnesses
- Driver runs each agent and answers its prompts
 - Fresh directory per task; checker verifies files
 - Measures success, tool calls, user interactions

Safety: agent self-correction

11 routine commands with hidden destructive effects			
L1 script	✓ cleanup	✓ _u deploy	✓ migration
L2 Makefile	✓ build	✓ _u install	✓ formatter
L3 chain	✓ test-runner	✓ build-pkg	
L∞ binary	✓ lint	✓ _u config-fix	✓ optimizer
8 / 11 self-corrected; ✓ _u staged for the user. No baseline agent self-corrects.			

Example: formatter

- 🧑 Asks the agent to run the formatter (a Makefile that calls a script)
- 📁 YoloFS shows the effects: 2 source files rewritten, 2 docs **deleted**
- 🤖 "CRITICAL: This is a destructive script, not a legitimate formatter!"
→ yolo travel

Autonomy: fewer prompts

112 routine file operations

0.9 → 0.4 prompts per task vs. Claude Code, same 99% success

Codex 0.4 · Copilot 1.3 · Gemini 2.2

Performance

Linux kernel dev: YoloFS ≈ Ext4, OverlayFS +18%

